



## آزمایشگاه طراحی سامانه‌های رقیمی

تابستان ۱۴۰۵

دستیار استاد: علی اثنی‌عشری

مهلت ارسال: ۱۵ شهریور ماه

پیاده‌سازی و توسعه یک پردازنده ساده

آزمایش دهم

- مهلت ارسال پاسخ تمرین تا ساعت ۷:۵۹ تعیین شده است.
- با توجه به محدودیت زمانی آزمایشگاه، طراحی و کدنویسی هر آزمایش باید پیش از جلسه انجام شده و در قالب پیش‌گزارش تحویل داده شود. در جلسه آزمایشگاه فقط پیاده‌سازی طرح روی برد FPGA انجام می‌شود.
- بارم‌بندی کلی هر آزمایش به شرح زیر است: ۳۵ درصد نمره برای پیش‌گزارش، ۱۰ درصد برای حضور در کلاس، ۳۵ درصد برای پیاده‌سازی روی برد و ۲۰ درصد برای گزارش کار نهایی در نظر گرفته شده است. پیش‌گزارش باید شامل گزارش متنی کامل مراحل انجام آزمایش و پروژه نرم‌افزار Quartus با طراحی قابل سنتز باشد. نمره بخش پیاده‌سازی روی برد از آزمایش سوم به بعد اعمال می‌شود. گزارش کار نهایی نیز باید شامل نسخه نهایی و کامل پروژه، توضیحات دقیق درباره الگوریتم مورد استفاده و تصمیم‌های خاص اتخاذ شده به دلیل محدودیت‌های برد باشد؛ برای نمونه، محدودیت در وارد کردن یک عدد پنج‌رقمی BCD باید به‌طور کامل توضیح داده شود.
- در صورت غیبت دانشجو در یک جلسه، علاوه بر ۱۰ درصد نمره حضور، ۳۵ درصد نمره پیاده‌سازی نیز کسر خواهد شد؛ بنابراین، مجموع نمره کسر شده برابر با ۴۵ درصد است.
- در شرایط بسیار خاص، دانشجو می‌تواند با هماهنگی قبلی، جلسه غیبت شده را در جلسه‌ای دیگر جبران کند. در این حالت، نمایش و اجرای طرح روی برد FPGA الزامی است.
- پیش‌گزارشی که با تأخیر ارسال شود، فاقد ارزش نمره‌ای خواهد بود. همچنین، به‌ازای هر روز تأخیر در ارسال گزارش کار نهایی، ۳۰ درصد از نمره آن کسر می‌شود. گزارش کار نهایی حداکثر تا دو روز پس از مهلت تعیین شده قابل ارسال است.
- برای نگارش گزارش کار، حتماً از قالب **Course Project Report Template** موجود در [این پیوند](#) استفاده فرمایید.
- پاسخ‌های خود را به‌صورت تایپ‌شده ارسال کرده و حتماً نکات نگارشی را رعایت فرمایید.
- در سؤال‌هایی که نیازمند برنامه‌نویسی هستند، فایل کد کامنت‌گذاری شده و فایل آزمون را نیز همراه با گزارش ارسال فرمایید.
- تمام فایل‌های خود را در یک فایل فشرده با پسوند **zip** قرار دهید. نام فایل باید مطابق قالب زیر باشد:

Hw10\_[StudentId]\_[firstName]\_[lastName]

## شرح آزمایش دهم

یک پردازنده با معماری پشته‌ای در نظر بگیرید که دارای یک پشته شامل هشت ثبات ۸ بیتی است. این پردازنده دارای حافظه‌ای با ظرفیت ۲۵۶ خانه ۸ بیتی است.

هشت خانه آخر حافظه، یعنی آدرس‌های F8 تا FF، برای ارتباط با تجهیزات ورودی و خروجی به صورت Memory Mapped I/O در نظر گرفته شده‌اند.

در این آزمایش، مجموعه دستورهای پردازنده پایه باید با اضافه کردن سه دستور جدید توسعه داده شود. بنابراین پردازنده نهایی دارای ۱۱ دستور خواهد بود.

### قرارداد پشته

در توضیح دستورهای دو عملوندی، نمادهای زیر استفاده می‌شوند:

- مقدار بالاترین عنصر پشته یا Top of Stack است.
- مقدار دومین عنصر از بالای پشته است.
- در دستورهایی که دو مقدار از پشته حذف می‌شوند، ابتدا A و سپس B از پشته خوانده می‌شوند.
- نتیجه دستور SUB به صورت B-A محاسبه می‌شود.

### مجموعه دستورهای پردازنده

دستورهای پردازنده به همراه کدهای عملیاتی آن‌ها به شرح زیر هستند:

**C    PUSHC    0000** این دستور مقدار ثابت<sup>۱</sup> هشت‌بیتی C را در بالای پشته قرار می‌دهد.

**M    PUSH    0001** این دستور مقدار خانه حافظه یا درگاه ورودی با آدرس هشت‌بیتی M را خوانده و در بالای پشته قرار می‌دهد.

**M    POP    0010** این دستور مقدار بالای پشته را حذف کرده و آن را در خانه حافظه یا درگاه خروجی با آدرس M می‌نویسد.

**JUMP    0011** این دستور مقدار بالای پشته را حذف کرده و آن را در ثبات PC قرار می‌دهد. در نتیجه، اجرای برنامه به آدرس مشخص شده منتقل می‌شود.

**JZ    0100** این دستور آدرس پرش را از بالای پشته حذف می‌کند. اگر پرچم Z برابر یک باشد، آدرس خوانده شده در PC قرار می‌گیرد. در غیر این صورت، اجرای برنامه از دستور بعدی ادامه پیدا می‌کند.

**JS    0101** این دستور آدرس پرش را از بالای پشته حذف می‌کند. اگر پرچم S برابر یک باشد، آدرس خوانده شده در PC قرار می‌گیرد. در غیر این صورت، اجرای برنامه از دستور بعدی ادامه پیدا می‌کند.

<sup>۱</sup>Constant

**ADD 0110** این دستور دو مقدار بالای پشته، یعنی A و B، را حذف کرده و مقدار B+A را محاسبه می‌کند. سپس نتیجه هشت‌بیتی را در بالای پشته قرار می‌دهد.

این دستور پرچم‌های S و Z را براساس نتیجه محاسبه‌شده به‌روزرسانی می‌کند.

**SUB 0111** این دستور دو مقدار بالای پشته، یعنی A و B، را حذف کرده و مقدار

$$B - A$$

را محاسبه می‌کند. سپس نتیجه هشت‌بیتی را در بالای پشته قرار می‌دهد.

این دستور پرچم‌های S و Z را براساس نتیجه محاسبه‌شده به‌روزرسانی می‌کند.

**SWAP 1000** این دستور جای دو عنصر بالای پشته را عوض می‌کند. اگر وضعیت پشته پیش از اجرای دستور به‌صورت زیر باشد:

$$\text{Top} \rightarrow A, B, \dots$$

پس از اجرای دستور، وضعیت پشته به‌صورت زیر خواهد بود:

$$\text{Top} \rightarrow B, A, \dots$$

اجرای این دستور تغییری در پرچم‌های S و Z ایجاد نمی‌کند.

**SHL 1001** این دستور مقدار بالای پشته را یک بیت به سمت چپ شیفت می‌دهد. بیت کم‌ارزش نتیجه برابر صفر قرار گرفته و بیت پرارزش قبلی حذف می‌شود.

بنابراین، در صورت نبود سرریز، این دستور معادل ضرب مقدار بالای پشته در عدد دو است:

$$\text{Top} \leftarrow (\text{Top} \ll 1)$$

نتیجه به‌صورت هشت‌بیتی در همان محل بالای پشته قرار می‌گیرد. اجرای این دستور تغییری در پرچم‌های S و Z ایجاد نمی‌کند.

**CMP 1010** این دستور دو مقدار بالای پشته را بدون حذف‌کردن آن‌ها با یکدیگر مقایسه می‌کند.

برای انجام مقایسه، مقدار موقت زیر محاسبه می‌شود:

$$T = B - A$$

مقدار T در پشته ذخیره نمی‌شود و ترتیب و محتوای پشته پس از اجرای دستور بدون تغییر باقی می‌ماند.

پرچم‌ها براساس مقدار T به صورت زیر به روزرسانی می شوند:

$$Z = \begin{cases} 1 & T = 0, \\ 0 & T \neq 0, \end{cases}$$

$$S = T_7.$$

در نتیجه:

- اگر  $B=A$  باشد، پرچم Z برابر یک خواهد شد.
  - اگر  $B<A$  باشد، پرچم S برابر یک خواهد شد.
  - اگر  $B>A$  باشد، هر دو پرچم S و Z برابر صفر خواهند شد.
- کدهای عملیاتی 1011 تا 1111 در این آزمایش رزرو شده هستند و نباید مورد استفاده قرار گیرند.

## پرچم‌های پردازنده

پردازنده فقط دارای دو پرچم S و Z است. پرچم Z در صورتی برابر یک می شود که نتیجه محاسبه برابر صفر باشد. پرچم S برابر بیت پرارزش نتیجه هشت بیتی است و منفی بودن نتیجه را در نمایش مکمل دو مشخص می کند. تنها دستورهای زیر مجاز به تغییر پرچم‌ها هستند:

ADD      SUB      CMP

اجرای تمام دستورهای دیگر، از جمله PUSH، POP، JUMP، JZ، JS، SWAP و SHL، نباید مقدار پرچم‌های S و Z را تغییر دهد. تمام محاسبات پردازنده هشت بیتی، علامت دار و مبتنی بر نمایش مکمل دو هستند. بیت شماره هفت هر داده، بیت علامت آن است.

## نگاشت ورودی و خروجی

ارتباط پردازنده با کلیدها، LEDها و نمایشگرهای Seven Segment باید فقط از طریق Memory Mapped I/O انجام شود. نگاشت آدرس‌های ورودی و خروجی به صورت زیر در نظر گرفته می شود:

| آدرس     | نوع دسترسی  | عملکرد                                                                                   |
|----------|-------------|------------------------------------------------------------------------------------------|
| F8       | فقط خواندنی | مقدار هشت‌بیتی ورودی X که از کلیدهای برد دریافت می‌شود.                                  |
| F9       | فقط نوشتنی  | رقم کم‌ارزش خروجی هگزادسیمال. تنها چهار بیت کم‌ارزش داده نوشته‌شده در نظر گرفته می‌شوند. |
| FA       | فقط نوشتنی  | رقم پرارزش خروجی هگزادسیمال. تنها چهار بیت کم‌ارزش داده نوشته‌شده در نظر گرفته می‌شوند.  |
| FB       | فقط نوشتنی  | خروجی خطا. بیت صفر این درگاه به LED خطا متصل است.                                        |
| FC       | فقط نوشتنی  | خروجی پایان محاسبه. بیت صفر این درگاه به LED پایان عملیات متصل است.                      |
| FD تا FF | رزروده      | این آدرس‌ها در این آزمایش استفاده نمی‌شوند.                                              |

برای نمایش خروجی، نیم‌بایت کم‌ارزش Y باید در آدرس F9 و نیم‌بایت پرارزش آن باید در آدرس FA نوشته شود. بنابراین مقدار خروجی روی دو واحد Seven Segment به‌صورت هگزادسیمال نمایش داده خواهد شد.

## صورت مسئله

ابتدا سه دستور جدید SWAP، SHL و CMP را به مسیر داده و واحد کنترل پردازنده اضافه کنید. سپس با استفاده از پردازنده توسعه‌یافته، یک برنامه زبان ماشین بنویسید که مقدار هشت‌بیتی X را از آدرس F8 دریافت کرده و تابع چندبخشی زیر را محاسبه کند:

$$Y = \begin{cases} 3X + 5, & 0 \leq X < 32, \\ 2X + 20, & 32 \leq X < 64, \\ X - 40, & 64 \leq X \leq 127. \end{cases}$$

تشخیص بازه‌ای که X در آن قرار دارد باید با استفاده از دستور CMP و پرش‌های شرطی انجام شود. در محاسبه عبارت‌های  $2X$  و  $3X$ ، استفاده از عملگر ضرب سخت‌افزاری یا علامت \* در کد توصیف سخت‌افزار مجاز نیست. مقدار  $2X$  باید با استفاده از دستور SHL محاسبه شود. برای محاسبه  $3X$  می‌توان از رابطه زیر استفاده کرد:

$$3X = X + 2X.$$

در صورت نیاز، مقدار ورودی X می‌تواند ابتدا در یکی از خانه‌های عادی حافظه ذخیره شده و سپس چند بار از حافظه خوانده شود.

## شرایط خطا

در هریک از شرایط زیر، نتیجه نامعتبر محسوب می شود:

۱. ورودی X یک عدد منفی باشد؛ یعنی بیت شماره هفت آن برابر یک باشد.

۲. مقدار ریاضی خروجی Y بزرگتر از ۱۲۷ شود.

در صورت وقوع خطا، برنامه باید عملیات زیر را انجام دهد:

- مقدار یک را در آدرس FB بنویسد تا LED خطا روشن شود.
- مقدار صفر را در آدرس های F9 و FA بنویسد تا خروجی قبلی از روی نمایشگرها پاک شود.
- مقدار یک را در آدرس FC بنویسد تا پایان عملیات مشخص شود.
- در صورت معتبر بودن خروجی، برنامه باید عملیات زیر را انجام دهد:
- مقدار صفر را در آدرس FB بنویسد تا LED خطا خاموش باشد.
- نیم بایت کم ارزش Y را در آدرس F9 بنویسد.
- نیم بایت پر ارزش Y را در آدرس FA بنویسد.
- مقدار یک را در آدرس FC بنویسد تا پایان محاسبه مشخص شود.

توجه کنید که در بازه دوم تابع، برای ورودی های  $X=54$  تا  $X=63$ ، خروجی ریاضی بزرگتر از ۱۲۷ خواهد بود و باید به عنوان خطا تشخیص داده شود.

## الزامات پیاده سازی

موارد زیر باید در طراحی و گزارش ارائه شوند:

۱. مسیر داده و واحد کنترل پردازنده باید برای پشتیبانی از دستورهای SWAP، SHL و CMP توسعه داده شوند.
۲. تمام بخش های طراحی باید قابل سنتز باشند.
۳. برنامه زبان ماشین باید از دستور CMP برای تشخیص بازه های تابع استفاده کند.
۴. برنامه زبان ماشین باید از دستور SHL برای محاسبه ضرب در دو استفاده کند.
۵. برای دستور SWAP باید آزمون مستقل در Testbench ارائه شود؛ حتی اگر استفاده از آن در برنامه نهایی ضروری نباشد.
۶. استفاده مستقیم از عملگر ضرب برای پیاده سازی تابع مجاز نیست.
۷. ارتباط با ورودی، نمایشگرها و LED ها باید فقط از طریق آدرس های Memory Mapped I/O مشخص شده انجام شود.

۸. حداکثر عمق استفاده شده از پشته در طول اجرای برنامه باید در گزارش مشخص شود.

۹. محتوای حافظه برنامه باید به صورت یک جدول شامل آدرس، کد ماشین، نام دستور و توضیح عملکرد ارائه شود.

۱۰. فلوچارت برنامه زبان ماشین باید در گزارش قرار داده شود.

## آزمون‌های الزامی

فایل Testbench باید حداقل ورودی‌های زیر را بررسی کند:

| X   | خروجی مورد انتظار | وضعیت خطا | دلیل انتخاب ورودی                                  |
|-----|-------------------|-----------|----------------------------------------------------|
| 0   | 5                 | بدون خطا  | کوچک‌ترین ورودی معتبر و مسیر اول تابع.             |
| 31  | 98                | بدون خطا  | آخرین مقدار بازه اول.                              |
| 32  | 84                | بدون خطا  | اولین مقدار بازه دوم.                              |
| 53  | 126               | بدون خطا  | بزرگ‌ترین خروجی معتبر در بازه دوم.                 |
| 54  | 128               | خطا       | اولین ورودی که خروجی آن بزرگ‌تر از ۱۲۷ است.        |
| 63  | 146               | خطا       | آخرین مقدار بازه دوم و بررسی خروجی خارج از محدوده. |
| 64  | 24                | بدون خطا  | اولین مقدار بازه سوم.                              |
| 127 | 87                | بدون خطا  | بزرگ‌ترین ورودی مثبت هشت‌بیتی.                     |
| 128 | نامعتبر           | خطا       | کوچک‌ترین الگوی هشت‌بیتی دارای علامت منفی.         |
| 255 | نامعتبر           | خطا       | نمایش مکمل دوی عدد منفی یک.                        |

علاوه بر آزمون برنامه کامل، عملکرد هرکدام از دستورهای جدید باید به صورت مستقل بررسی شود.

برای دستور SWAP، حداقل یک آزمون با دو مقدار متفاوت در بالای پشته ارائه شود.

برای دستور SHL، حداقل مقادیر 0، 1، 63 و 127 بررسی شوند.

برای دستور CMP، هر سه حالت زیر باید آزموده شوند:

$$B < A, \quad B = A, \quad B > A.$$

## موارد مورد انتظار در پیش‌گزارش

پیش‌گزارش باید حداقل شامل موارد زیر باشد:

۱. نمودار بلوکی پردازنده توسعه‌یافته؛
۲. توضیح تغییرات مسیر داده برای سه دستور جدید؛
۳. توضیح تغییرات واحد کنترل؛
۴. جدول ریز عملیات هر دستور جدید؛
۵. فلوچارت برنامه محاسبه تابع چندبخشی؛
۶. جدول کامل کد ماشین برنامه؛
۷. محاسبه حداکثر عمق پشته؛
۸. فایل‌های قابل سنتز پروژه Quartus؛
۹. فایل Testbench برای پردازنده و برنامه نهایی.